



Class Notes: Object-Oriented Programming Concepts (Sessions 3 & 4) - Java

Session 3: Abstraction

Duration: 2 hours

Objective: Understand abstraction, its role in OOP, and how to implement abstract classes and interfaces in Java.

Overview

- **What is Abstraction?:** Abstraction is the process of hiding implementation details and exposing only essential features to the user. It simplifies complex systems by focusing on what an object does rather than how it does it.
- **Abstraction vs. Encapsulation:**
 - **Abstraction:** Hides complexity (e.g., a car's interface shows speed but not engine mechanics).
 - **Encapsulation:** Hides data and controls access (e.g., private fields with getters/setters).
- **Tools for Abstraction in Java:**
 - **Abstract Classes:** Classes declared with the `abstract` keyword, which may contain abstract methods (without implementation) and concrete methods.
 - **Interfaces:** Purely abstract constructs that define methods without implementation, allowing multiple inheritance-like behavior.
- **Forward Declaration:** Not applicable in Java (common in C++ for header files); Java uses interfaces or abstract classes for similar purposes.
- **Benefits:**
 - Simplifies system design.
 - Enhances flexibility by allowing implementation changes without affecting users.

Activity 1: Brainstorming Abstraction (15 minutes)

- **Task:** In groups of 3–4, identify a real-world system (e.g., a vending machine or ATM) and discuss how abstraction can simplify its interface. List one abstract class or interface with its methods.
- **Example:**
 - System: Vending Machine
 - Interface: `Vendable`
 - Methods: `selectItem()` , `dispenseItem()`
 - Outcome: Groups present their interface/abstract class, reinforcing abstraction's role in hiding complexity.

Use Case: Vehicle Management System

- **Scenario:** A rental company manages different vehicles (cars, bikes).
- **OOP Approach in Java:**
 - Abstract Class: `Vehicle` with abstract method `start()` .
 - Concrete Classes: `Car` , `Bike` implementing `start()` .
 - Interface: `Rentable` with methods like `calculateRentalCost()` .
- **Benefit:** Abstraction allows users to interact with vehicles without knowing specific engine details.

Example 1: Abstract Class and Interface in Java

```

// Abstract class
abstract class Vehicle {
    protected String model;
    public Vehicle(String model) {
        this.model = model;
    }
    // Abstract method
    abstract void start();
    // Concrete method
    public void display() {
        System.out.println("Model: " + model);
    }
}

// Interface
interface Rentable {
    double calculateRentalCost(int days);
}

// Concrete class implementing abstract class and interface
class Car extends Vehicle implements Rentable {
    public Car(String model) {
        super(model);
    }
    @Override
    void start() {
        System.out.println(model + " car started with key ignition.");
    }
    @Override
    public double calculateRentalCost(int days) {
        return days * 50.0; // $50 per day
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car("Toyota Camry");
        car.display();
        car.start();
    }
}

```

```
        System.out.println("Rental cost for 3 days: $" + car.calculateRentalCost(3));  
    }  
}
```

- **Output:**

```
Model: Toyota Camry  
Toyota Camry car started with key ignition.  
Rental cost for 3 days: $150.0
```

- **Explanation:** The `Vehicle` abstract class defines a common structure, while `Rentable` ensures rental functionality. `Car` provides specific implementations.

Classwork 1: Implement Abstraction (20 minutes)

- **Task:** In pairs, write a Java program to define an abstract class `Shape` with an abstract method `calculateArea()` and a concrete method `describe()`. Create two concrete classes, `Circle` and `Rectangle`, implementing `calculateArea()`.
- **Instructions:**
 - i. Define the abstract class and interface (if desired).
 - ii. Implement the concrete classes with appropriate calculations.
 - iii. Test with two objects of different shapes.
- **Sample Solution:**

```
abstract class Shape {  
    protected String name;  
    public Shape(String name) {  
        this.name = name;  
    }  
    abstract double calculateArea();  
    public void describe() {  
        System.out.println("This is a " + name + " with area " + calculateArea());  
    }  
}
```

```
class Circle extends Shape {  
    private double radius;  
    public Circle(String name, double radius) {  
        super(name);  
        this.radius = radius;  
    }  
    @Override  
    double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

```
class Rectangle extends Shape {  
    private double length, width;  
    public Rectangle(String name, double length, double width) {  
        super(name);  
        this.length = length;  
        this.width = width;  
    }  
    @Override  
    double calculateArea() {  
        return length * width;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Shape circle = new Circle("Circle", 5.0);  
    }  
}
```

```
Shape rectangle = new Rectangle("Rectangle", 4.0, 6.0);
circle.describe();
rectangle.describe();
}
}
```

- **Output:**

```
This is a Circle with area 78.53981633974483
This is a Rectangle with area 24.0
```

Session 4: Inheritance

Duration: 2 hours

Objective: Understand inheritance, its types, and how to implement it in Java, including constructor execution.

Overview

- **What is Inheritance?:** A mechanism where a class (subclass) inherits fields and methods from another class (superclass), promoting code reuse.
- **Types of Inheritance:**
 - **Single:** One class inherits from one superclass (e.g., `Car` extends `Vehicle`).
 - **Multilevel:** A chain of inheritance (e.g., `SportsCar` extends `Car` extends `Vehicle`).
 - **Hierarchical:** Multiple classes inherit from one superclass (e.g., `Car` and `Bike` extend `Vehicle`).
 - **Multiple:** Not supported in Java for classes (to avoid diamond problem); achieved via interfaces.
- **Constructor Execution:** Subclass constructors implicitly or explicitly call superclass constructors using `super()` .
- **Benefits:**
 - Code reusability.
 - Hierarchical organization of classes.
- **Challenges:** Overuse can lead to complex hierarchies; Java avoids multiple inheritance issues with interfaces.

Activity 2: Inheritance Mapping (15 minutes)

- **Task:** Individually, design a class hierarchy for an animal shelter system. Identify a superclass (e.g., `Animal`) and at least two subclasses (e.g., `Dog` , `Cat`). List inherited and unique fields/methods.
- **Example:**
 - Superclass: `Animal` (fields: `name` , `age` ; method: `makeSound()`)
 - Subclass: `Dog` (unique method: `fetch()`)
 - Subclass: `Cat` (unique method: `purr()`)
- **Outcome:** Students share hierarchies, reinforcing inheritance concepts.

Use Case: Employee Management System

- **Scenario:** A company manages different types of employees (e.g., full-time, part-time).
- **OOP Approach in Java:**
 - Superclass: `Employee` with fields `name` , `id` and method `calculateSalary()` .
 - Subclasses: `FullTimeEmployee` , `PartTimeEmployee` overriding `calculateSalary()` .
- **Benefit:** Inheritance allows shared functionality (e.g., ID management) while customizing salary calculations.

Example 2: Inheritance in Java

```
class Employee {
    protected String name;
    protected int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public double calculateSalary() {
        return 0.0; // Base salary
    }

    public void display() {
        System.out.println("Name: " + name + ", ID: " + id + ", Salary: $" + calculateSalary());
    }
}

class FullTimeEmployee extends Employee {
    private double monthlySalary;

    public FullTimeEmployee(String name, int id, double monthlySalary) {
        super(name, id);
        this.monthlySalary = monthlySalary;
    }

    @Override
    public double calculateSalary() {
        return monthlySalary;
    }
}

class PartTimeEmployee extends Employee {
    private double hourlyRate;
    private int hoursWorked;

    public PartTimeEmployee(String name, int id, double hourlyRate, int hoursWorked) {
        super(name, id);
        this.hourlyRate = hourlyRate;
    }
}
```

```

        this.hoursWorked = hoursWorked;
    }

    @Override
    public double calculateSalary() {
        return hourlyRate * hoursWorked;
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp1 = new FullTimeEmployee("Alice", 101, 5000.0);
        Employee emp2 = new PartTimeEmployee("Bob", 102, 20.0, 80);
        emp1.display();
        emp2.display();
    }
}

```

- **Output:**

```

Name: Alice, ID: 101, Salary: $5000.0
Name: Bob, ID: 102, Salary: $1600.0

```

- **Explanation:** `Employee` provides common fields and methods, while subclasses override `calculateSalary()` to provide specific logic. `super()` ensures proper constructor chaining.

Classwork 2: Implement Inheritance (25 minutes)

- **Task:** In groups of 2–3, write a Java program to define a superclass `Animal` with an abstract method `makeSound()`. Create two subclasses, `Dog` and `Cat`, with specific sound implementations and additional unique methods.
- **Instructions:**
 - i. Use `abstract` for the superclass method.
 - ii. Include a constructor in `Animal` and use `super()` in subclasses.
 - iii. Test with two objects of different animals.
- **Sample Solution:**

```
abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    abstract void makeSound();

    public void display() {
        System.out.println("Animal: " + name);
    }
}

class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " says: Woof!");
    }

    public void fetch() {
        System.out.println(name + " is fetching the ball.");
    }
}

class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " says: Meow!");
    }
}
```

```

    public void purr() {
        System.out.println(name + " is purring.");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog("Rex");
        Animal cat = new Cat("Luna");
        dog.display();
        dog.makeSound();
        ((Dog) dog).fetch();
        cat.display();
        cat.makeSound();
        ((Cat) cat).purr();
    }
}

```

- **Output:**

```

Animal: Rex
Rex says: Woof!
Rex is fetching the ball.
Animal: Luna
Luna says: Meow!
Luna is purring.

```

Scenario: Online Learning Platform

- **Context:** A platform manages courses and their types (e.g., video-based, text-based).
- **OOP Design in Java:**
 - Abstract Class: `Course` with abstract method `deliverContent()`.
 - Subclasses: `VideoCourse`, `TextCourse` implementing `deliverContent()`.
 - Interface: `Certifiable` with method `issueCertificate()`.
- **Activity:** Discuss how abstraction hides delivery details and inheritance allows shared course properties (e.g., `title`, `duration`).

Class Test: Sessions 3 & 4 (30 minutes)

- **Instructions:** Answer all questions. Write code in Java. Submit answers on paper or via Onlinevarsity.
- **Questions:**
 - i. **Conceptual (5 points):**
 - Define abstraction and explain how it differs from encapsulation. (2 points)
 - List three types of inheritance supported in Java and briefly describe them. (3 points)
 - ii. **Short Answer (5 points):**
 - Explain the role of the `super` keyword in Java inheritance with an example. (3 points)
 - What is the purpose of an interface in Java? Provide a real-world analogy. (2 points)
 - iii. **Coding (10 points):**
 - Write a Java program to define an abstract class `Appliance` with a private field `brand` and an abstract method `operate()`. Create two subclasses, `WashingMachine` and `Refrigerator`, with specific implementations of `operate()` and a constructor that initializes `brand`. Create two objects and demonstrate their use. (10 points)
- **Sample Solution for Question 3 (Java):**

```
abstract class Appliance {
    private String brand;

    public Appliance(String brand) {
        this.brand = brand;
    }

    public String getBrand() {
        return brand;
    }

    abstract void operate();
}

class WashingMachine extends Appliance {
    public WashingMachine(String brand) {
        super(brand);
    }

    @Override
    void operate() {
        System.out.println(getBrand() + " washing machine is washing clothes.");
    }
}

class Refrigerator extends Appliance {
    public Refrigerator(String brand) {
        super(brand);
    }

    @Override
    void operate() {
        System.out.println(getBrand() + " refrigerator is cooling food.");
    }
}

public class Main {
    public static void main(String[] args) {
        Appliance washer = new WashingMachine("Samsung");
    }
}
```

```
        Appliance fridge = new Refrigerator("LG");
        washer.operate();
        fridge.operate();
    }
}
```

- **Output:**

```
Samsung washing machine is washing clothes.
LG refrigerator is cooling food.
```

- **Grading Criteria:**

- Conceptual: Clarity and correctness (5 points).
- Short Answer: Accuracy and relevance (5 points).
- Coding: Correct syntax, abstraction, inheritance, and functionality (10 points).

Additional Notes

- **Resources** (Page 5):

- Use Onlinevarsity for eBook access, glossary, FAQs, and practice tests.
- Submit lab assignments (e.g., Classwork 1 and 2) via Atrack for faculty evaluation.

- **References** (Page 11):

- *Sams Teach Yourself Object Oriented Programming in 21 Days* by Anthony Sintes.
- *An Introduction To Object Oriented Programming, 3/e* by Timothy Budd.

- **Feedback:** Submit feedback on course material via Onlinevarsity's feedback feature.

- **Next Steps:** Review classwork and test submissions. Prepare for Session 5 on multiple inheritance and interfaces by reading relevant eBook sections.